OPT Training: Energy & Resource Profiling

Measuring overhead, energy consumption, and GPU utilization across batch sizes 4, 8, and 16

Presenter: Aly Mohamed

Workload Overview



Model

OPT
(Open Pre-trained Transformer)



Dataset

Synthetic data
(100,000 samples)



Setup

Slurm cluster

Batch Sizes Tested

4	8	16

Each experiment: 5 minutes × 3 runs, metrics averaged

Experiment Profiles: What Each One Measures

Profile A

`trainer_stats: noop`

Baseline – Wall-Clock Time Only

⏱️ Zero instrumentation overhead — training loop runs as-is

📏 Records only end-to-end wall-clock duration

🎯 Used as the reference to compute overhead % of B and C

🚫 No energy, no resource, no phase timing collected

Profile B

`trainer_stats: codecarbon`

End-to-End Energy (CodeCarbon only)

⚡ One OfflineEmissionsTracker wraps the full training session

📡 CodeCarbon samples GPU power via NVML every ~500 ms

🔢 Step-level tracking disabled (track_steps=0) to minimise overhead

Profile C

`trainer_stats: composite(codecarbon + resource)`

Fine-Grained: Timing + Resources + Energy

🔬 simple stats: per-phase timers (forward / backward / optimizer)

📄 resource stats: GPU util, GPU mem, CPU, power via NVML — every step

⚡ Energy sampled every 500 ms; per-step energy inferred from counters

📡 CodeCarbon samples GPU power via NVML every ~500 ms

📊 Overhead vs A shows cost of energy monitoring alone (~1.5%)

Each profile × batch size combination is run 3 times · Overhead compliance threshold: <5% vs. Profile A

Training Loop: Warmup & Measurement

⏱ **Warmup Period = few steps** → **Measurement Period (max_duration_sec = 300 s)**

Each Training Step

1 forward()

Model under torch.cuda.amp.autocast (FP16 compute, FP32 gradients). Zeros gradients before pass.

2 backward()

loss.backward() — computes gradients. Monitors for NaN/Inf. GPU sync before/after for timing.

3 optimizer_step()

AdamW weight update with linear LR schedule. Monitors for NaN/Inf weights.

4 Stats hooks

Timing, resource sampling, and energy counters wrap each phase. Data logged per step to CSV.

torch.cuda.synchronize() called before/after each phase — ensures GPU work is complete before the timer stops

Controlling Experiments: CLI Configuration

All parameters are controlled via dot-notation CLI arguments that map to hierarchical config classes. Sub-configs are auto-discovered — no core files need to be modified to add new components.

Data	`--data_configs.opt.batch_size`	`4 / 8 / 16`	Number of sequences processed per training step. Max fitting power-of-2 = 16 for this model.
Data	`--data_configs.opt.seq_len`	`1024`	Token sequence length per sample. Kept fixed across all experiments for fair comparison.
Trainer	`--trainer_configs.simple.max_duration_sec`	`300`	Fixed-duration training window (5 minutes). Loop exits when elapsed time exceeds this — not epoch-based.

Sub-configs auto-discovered from src/config/{models,data,trainers,trainer_stats}/ · Fixed seed (1234) · Git commit hash logged per run

Measurement Overhead

How much does instrumentation slow training?

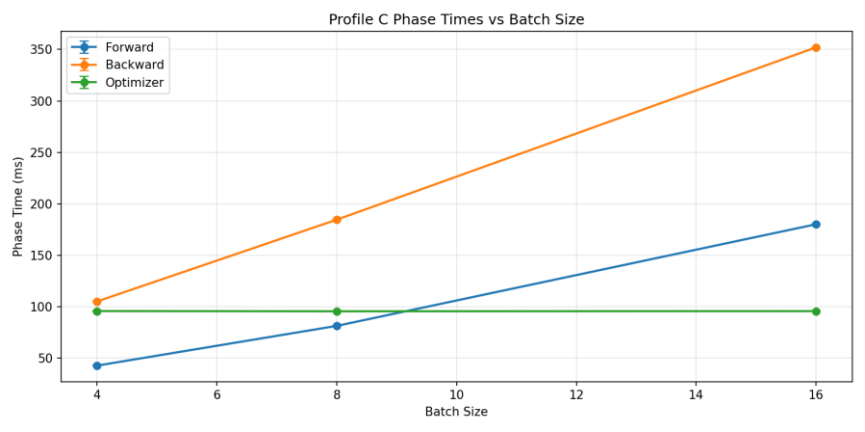
	Batch 4	Batch 8	Batch 16
Profiling overhead	1.49%	1.46%	1.61%

✓ **All overhead values well below the 5% compliance threshold**

Throughput & Phase Timing

4.05	2.74	1.58
steps/s	steps/s	steps/s
Batch 4	**Batch 8**	**Batch 16**

Larger batches → fewer steps/sec but more samples/sec (better GPU utilization per step)



Phase time vs. batch size

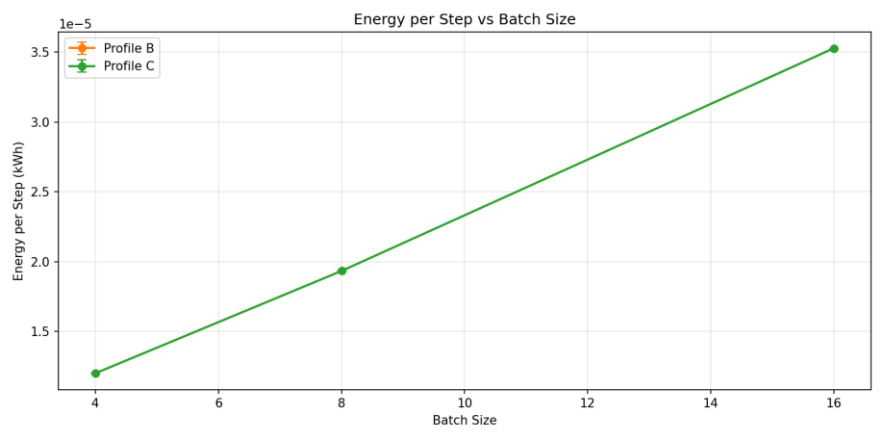
Energy Consumption

12.0
µWh/step
Batch 4

19.3
µWh/step
Batch 8

35.3
µWh/step
Batch 16

Larger batches consume more energy per step, but total run energy is similar (~5 min fixed window). Efficiency gain comes from more samples per joule.

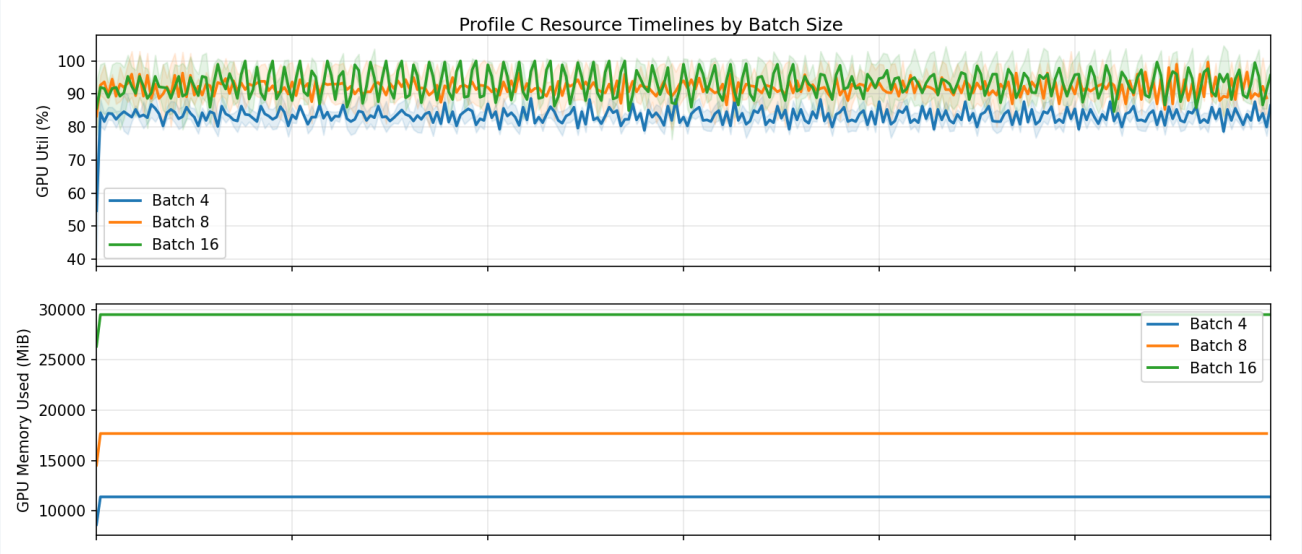


Energy per step vs. batch size

Total run energy: Batch 4 ≈ 14.6 mWh · Batch 8 ≈ 15.9 mWh · Batch 16 ≈ 16.7 mWh (all over ~5 min)

Profile B overhead (energy monitoring) adds ~1.5% run time — well within tolerance

GPU Efficiency Opportunities



83.3%

Batch 4

1 continuous window (entire run!)

Batch 16

91.8%

Batch 8

3 low-util windows

Batch 8

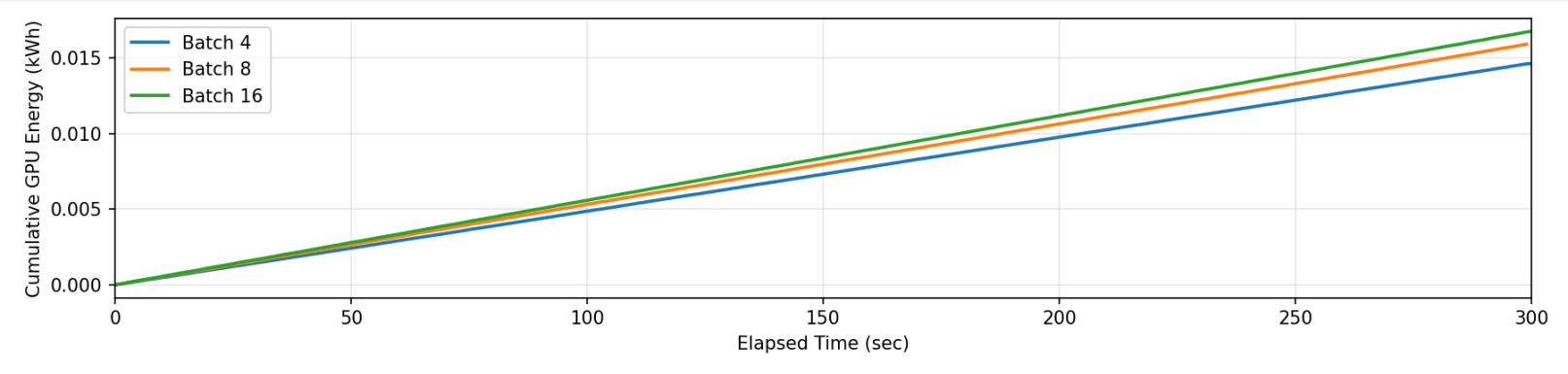
92.3%

Batch 16

29 short low-util windows

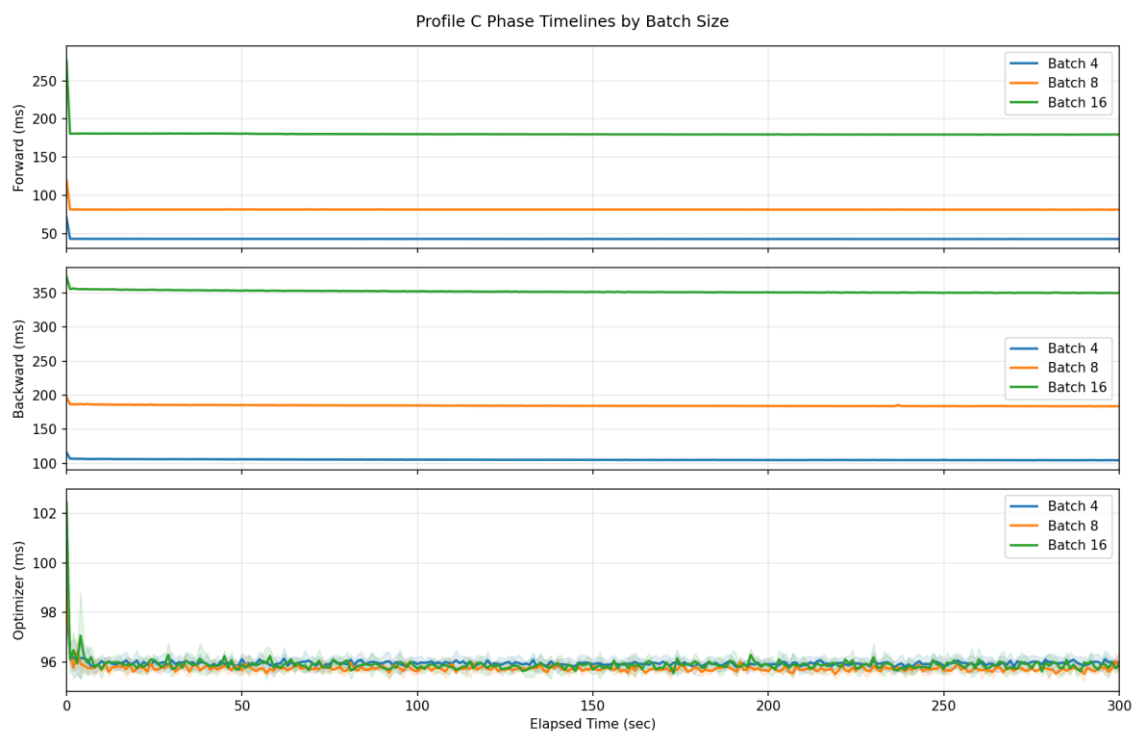
Batch 4

Resource Timelines – GPU Utilization, Memory & CPU



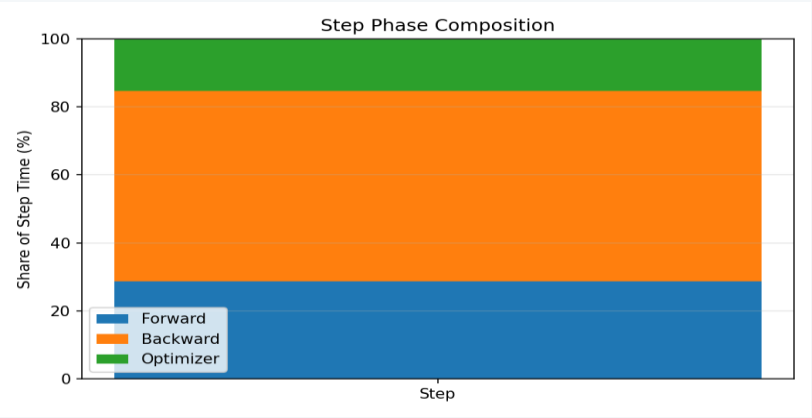
GPU util never reaches 100% · Batch 4 shows lowest avg util (~83%) · Memory scales linearly with batch size · CPU stays low (data already loaded)

Resource Timelines – Time per step



Per-Step Phase Breakdown

Phase	Batch 4	Batch 8	Batch 16	Trend
Forward pass	43.0 ms	81.6 ms	180.3 ms	Scales ~linearly with batch
Backward pass	105.3 ms	184.7 ms	352.1 ms	~2× forward time
Optimizer step	96.0 ms	95.7 ms	95.9 ms	Constant — param-count bound



Phase % of step time (batch 16)

Backward dominates at ~56% of step time
Optimizer: constant at ~96 ms regardless of batch
Forward + Backward scale together

The constant optimizer overhead is the primary opportunity for scheduling overlap (e.g., gradient compression, async communication)

Optimization opportunities

- Fused optimizer kernels to eliminate constant 96ms overhead
- Increase batch size
- Gradient checkpointing enables larger batches, less memory pressure
- Dedicated node removes shared Slurm contention noise

Measurement Quirks & Challenges

📊 Energy sampled at 500 ms — per-step inferred

CodeCarbon cannot sample faster than ~500 ms. Since steps are 240–630 ms, energy per step is inferred by dividing total energy by step count. This smooths out step-level variation but is sufficient for analysis.

🖥️ Shared Slurm cluster — resource contention

Experiments run on shared class infrastructure. Other jobs could affect measurements. Averaging over 3 runs mitigates noise; standard deviations were consistently small (<0.5s on duration), confirming reproducibility.

Summary & Takeaways

- ✅ All measurement overhead <2% — profiling is cheap for this workload
- 📈 Larger batch → higher energy per step
- 🔍 GPU utilization never reaches 100% — consistently ~83–93% across all batch sizes, indicating persistent efficiency headroom
- ⚙️ Backward pass dominates step time (~56%); optimizer step is constant ~96 ms independent of batch size
- ⚡ Batch 4 shows lowest GPU util (~83%) — optimizer overhead represents a higher fraction of total step time at small batches

Appendix

How We Measure: Libraries & Tools

CodeCarbon

v3.1.0

GPU power sampling via NVML → integrates over time → energy & carbon estimates

Metrics collected

- 3 OfflineEmissionsTracker instances per run (full / step / substep)
- Offline mode — no external API calls during training
- Location: Canada/Quebec grid carbon intensity
- Carbon = energy_kwh × carbon_intensity_gco2_per_kwh (default: 40 gCO2/kWh)
- Step tracking disabled in Profile B (track_steps=0) — one measurement only

nvidia-ml-py (NVML)

v13.580.82

Direct hardware queries to the GPU at each training step — no estimation

Metrics collected

- gpu_util (%) — GPU compute utilization
- gpu_mem_used / gpu_mem_total (MiB) — memory occupancy
- gpu_power_w (W) — instantaneous power draw
- gpu_energy_mj (mJ) — energy consumed during the step
- step_energy_kwh + cumulative_gpu_energy_kwh — per-step & running totals

psutil

v7.1.3

System & process resource monitoring — CPU, RAM, and I/O at each step

Metrics collected

- sys_cpu_percent (%) — overall system CPU load
- sys_mem_used / sys_mem_total (MiB) — system RAM
- proc_cpu_percent (%) — process-level CPU usage
- proc_rss / proc_vms (MiB) — process memory (resident/virtual)
- io_read / io_write (bytes) — disk I/O counters

Warmup period

The warmup is exactly 1 step (~0.77 seconds)

Step	Total (ms)	Forward (ms)	Backward (ms)	Optimizer (ms)
1	**760**	**284**	**374**	**102**
2	632	180	355	95
3+	~632	~180	~355	~95

GPU inefficiency

Optimizer is more significant for smaller batch size

Batch	Forward (ms)	Backward (ms)	Optimizer (ms)	Total step (ms)	Optimizer fraction
4	43	105	96	244	**39%**
8	82	185	96	362	**26%**
16	180	352	96	628	**15%**